

L2-Aware Pipeline Scheduling for Memory-Bound Multi-Cell GPU vRAN

Anonymous
NSDI '26 Submission

Abstract

5G virtualized radio access networks (vRAN) increasingly run physical-layer (PHY) processing on GPUs to meet sub-millisecond slot deadlines. Current deployments consolidate multiple cells onto a single GPU, expecting that compute parallelism will absorb the load. We show that this assumption is wrong. Through systematic profiling of the NVIDIA Aerial cu-PHY uplink pipeline on an A100 GPU, we find that the dominant kernel—channel equalization coefficient application—is memory-bound at an arithmetic intensity of 4.32 FLOP/byte, achieving only 0.86% of peak Tensor Core throughput even after FP16 optimization. The bottleneck is not compute but memory bandwidth: L1 cache throughput saturates at 93.6% in the scalar baseline and DRAM throughput rises to 21.2% after Tensor Core acceleration, leaving no headroom for concurrent cells.

When multiple cells share a GPU via CUDA streams, memory bandwidth contention causes per-cell latency degradation of up to $7.7\times$ at 8 concurrent cells—a superlinear growth that current schedulers, which allocate based on SM occupancy, cannot prevent. Through six controlled experiments on an A100, we establish three key findings: (1) the contention threshold is the L2 cache capacity (42 MB), not HBM bandwidth—two cells whose combined working set fits in L2 degrade only $1.12\times$, while cells exceeding L2 degrade $1.50\times$ or more; (2) pipeline stage ordering matters—launching the smaller-working-set kernel first reduces degradation from $1.50\times$ to $1.16\times$; (3) online contention prediction from just 5 runtime latency samples achieves $<3\%$ error, enabling adaptive batch sizing without offline profiling or hardware PMUs.

Based on these findings, we propose a research line for a memory-hierarchy-aware scheduler combining L2-aware stage co-scheduling, online contention prediction, and adaptive batch sizing with deadline guarantees. We present the experimental evidence, the contention model, and the development plan toward a full system targeting

NSDI.

1 Introduction

Cloud RAN centralizes baseband processing in data-center servers, replacing purpose-built radio hardware with software running on commodity accelerators. The promise is economic: pool processing resources across many cells, scale elastically with demand, and exploit commercial hardware economics. GPUs have emerged as the leading accelerator for this workload. NVIDIA Aerial [6] provides a CUDA-accelerated PHY stack that processes 5G NR uplink and downlink channels on GPUs, and production deployments use it to serve multiple cells from a single GPU.

The central question for multi-cell GPU vRAN is scheduling: how should kernels from multiple cells be assigned to GPU resources to meet the strict real-time deadlines of 5G? Each 5G NR slot must complete within 0.5 ms at 120 kHz subcarrier spacing; the uplink equalization pipeline alone must finish in a fraction of that budget. When a GPU serves one cell, the kernel runs in isolation and meets its deadline. When it serves four cells, the natural approach is to run all four concurrently using CUDA streams and rely on the GPU’s hardware scheduler to interleave them.

This approach fails because it treats the GPU as a compute-only resource. We profile the Aerial uplink equalization pipeline on an NVIDIA A100 and find that the dominant kernel—the coefficient application step of MMSE equalization—is memory-bound, not compute-bound. The kernel has an arithmetic intensity of 4.32 FLOP/byte, well below the A100’s Ridge point of ~ 161 FLOP/byte for FP16 Tensor Cores. Even after replacing the scalar FP32 inner loop with WMMA FP16 Tensor Core operations—a $5.5\times$ speedup that reduces latency from 0.146 ms to 0.027 ms—the kernel achieves only 2,681 GFLOP/s, or 0.86% of the 312 TFLOP/s Tensor Core peak. Profiling with Nsight Compute confirms

that the bottleneck is L1 cache throughput (93.6% of peak in the scalar version, 55.0% after Tensor Core optimization) and DRAM bandwidth (21.2% after optimization), not compute pipeline occupancy.

The implication for multi-cell scheduling is direct: if a single cell’s equalization kernel already consumes a large fraction of memory bandwidth, concurrent execution of multiple cells will saturate the memory subsystem. Our measurements confirm this: per-cell latency degrades by $1.50\times$ at 2 cells, $2.43\times$ at 4 cells, and $3.97\times$ at 8 cells when all cells run concurrently on separate CUDA streams. SM-based scheduling—which assigns cells to SM partitions or relies on the GPU’s warp scheduler to interleave—does not address this because the contention is in the memory hierarchy, not in the SMs.

This paper. We present a systematic experimental study of multi-cell memory contention in GPU vRAN and propose a research line for a memory-hierarchy-aware scheduler. Through six controlled experiments on an A100, we make three key findings that distinguish vRAN memory contention from the well-studied DNN inference case:

- **L2 cache capacity, not HBM bandwidth, is the contention threshold** (§4). Two 100 MHz cells with a combined working set of 33 MB (fitting in the 42 MB L2) degrade only $1.12\times$ when paired with a small 5 MHz cell, but $1.52\times$ when paired with another 100 MHz cell that overflows L2. This is fundamentally different from DNN serving, where working sets far exceed cache and HBM is the sole bottleneck.
- **Pipeline stage ordering reduces contention** (§4). The vRAN pipeline has three stages with different working set sizes (Gram: 13.8 MB, Coef: 16.5 MB, LLR: 5.6 MB). Launching the smaller-working-set stage first reduces degradation from $1.50\times$ (same-stage) to $1.16\times$ (cross-stage with ordering), because the smaller kernel occupies less L2, leaving capacity for the larger kernel.
- **Online contention prediction is feasible without PMUs** (§4). Just 5 runtime latency samples predict actual contention with $<3\%$ error for $N \geq 2$ cells. This enables adaptive batch sizing: the scheduler starts with a large batch, measures contention, and splits if degradation exceeds a threshold ($2.0\times$).

Based on these findings, we propose a scheduler design combining: (1) L2-aware stage co-scheduling that orders pipeline stages by working set size, (2) online contention prediction from timing-only measurements, and (3) adaptive batch sizing with deadline guarantees. We present the experimental evidence, the calibrated contention model, and the development plan toward a full system evaluation (§5–§7).

2 Background

2.1 5G NR Uplink PHY Processing

A 5G NR base station processes uplink signals through a pipeline of physical-layer functions. For PUSCH (Physical Uplink Shared Channel), the receiver pipeline includes: (1) FFT to convert time-domain samples to frequency-domain resource elements, (2) channel estimation from SRS (Sounding Reference Signal), (3) MMSE equalization combining channel estimates with received data, and (4) soft-demapping to produce log-likelihood ratios (LLRs) for the channel decoder.

The strict real-time constraint comes from the 5G NR slot structure. At 120 kHz subcarrier spacing (SCS), a slot spans 0.5 ms. The entire uplink pipeline—FFT, channel estimation, equalization, demapping, and decoding—must complete within this budget, with margin for downlink processing and fronthaul transport. Equalization alone is budgeted at most 100–200 μ s.

2.2 GPU-Accelerated vRAN

NVIDIA Aerial [6] implements the 5G PHY stack in CUDA, targeting data-center GPUs. The cuPHY library provides CUDA kernels for each pipeline stage. In production, a single GPU (e.g., A100, L40S) serves multiple cells concurrently: each cell’s pipeline is dispatched as a sequence of CUDA kernels on separate CUDA streams, and the GPU’s hardware scheduler interleaves execution across SMs.

This multi-cell consolidation is the economic driver for GPU vRAN. A single A100 has 108 SMs and 312 TFLOP/s of FP16 Tensor Core throughput—far exceeding one cell’s compute demand. The implicit assumption is that compute headroom translates directly into cell capacity: if one cell uses 10% of compute, ten cells should fit.

2.3 GPU Memory Hierarchy and the Roofline Model

An A100 GPU has a memory hierarchy consisting of: (1) per-SM shared memory/L1 cache (192 KB/SM, 20.7 TB/s aggregate), (2) L2 cache (40 MB, 5 TB/s), and (3) HBM2e DRAM (80 GB, 1.94 TB/s). The Roofline model [7] relates achievable performance to arithmetic intensity ($AI = \text{FLOPs/byte}$): a kernel is memory-bound if its AI is below the GPU’s Ridge point ($AI_{\text{ridge}} = \text{peak FLOP/s} \div \text{peak bandwidth}$). For FP16 Tensor Cores on A100, $AI_{\text{ridge}} = 312 \text{ TFLOP/s} \div 1.94 \text{ TB/s} \approx 161 \text{ FLOP/byte}$. Kernels with AI well below this value are memory-bound: their performance is limited by how

fast data can be moved, not how fast compute can execute.

3 Characterization: GPU vRAN is Memory-Bound

This section presents our profiling methodology and key findings: the dominant kernel in 5G uplink equalization is memory-bound, and this is structural across all realistic configurations.

3.1 Methodology

We profile the NVIDIA Aerial cuPHY PUSCH receiver pipeline on an NVIDIA A100 80GB PCIe (108 SMs, SM 8.0, 312 TFLOP/s FP16 TC peak, 1.94 TB/s HBM2e). We use Nsight Compute (NCU) for per-kernel metrics and CUDA events for end-to-end latency. All measurements use the reference configuration: 132 PRBs (100 MHz), 12 subcarriers/PRB, 8 MIMO layers, 11 data symbols, 64 base station antennas, yielding 1,584 subcarriers. Each latency measurement is the median of 500 alternating iterations after 50 warmup runs.

3.2 Bottleneck Identification

We first identify which kernel dominates the equalization pipeline. Table 1 shows the three stages of the PUSCH equalization pipeline and their measured latency.

Table 1: PUSCH equalization pipeline breakdown (100 MHz, 64 ant, 8 layers).

Kernel	Latency	Share	AI
Gram matrix	0.035 ms	19%	3.76
Coef. apply	0.146 ms	81%	4.32
LLR demap	~0.030 ms	~10%	low

The coefficient application kernel—which computes $Y_{eq}[sc, \ell, s] = \sum_a C[sc, \ell, a] \cdot Y_{rx}[sc, s, a]$ as a batched complex GEMM over 1,584 subcarriers—consumes 81% of equalization time. Despite the LLR soft-demapper being the perceived bottleneck (due to its constant-memory LUT stalls), profiling reveals that the coefficient application is the true dominant cost.

3.3 Tensor Core Optimization and the Memory Wall

The coefficient application has an underlying GEMM structure: per subcarrier, it computes $[8 \times 64] \times [64 \times 11]$ in complex arithmetic. We replace the scalar FP32 inner loop (64 serial complex FMAs per thread) with WMMA

FP16 Tensor Core operations using the standard complex decomposition into four real matrix multiplications:

$$\text{Re}(Y_{eq}) = \text{Re}(C) \cdot \text{Re}(Y) - \text{Im}(C) \cdot \text{Im}(Y)$$

$$\text{Im}(Y_{eq}) = \text{Re}(C) \cdot \text{Im}(Y) + \text{Im}(C) \cdot \text{Re}(Y)$$

Each warp processes one subcarrier: 4 k-steps $\times$ 4 real MMAs = 16 MMA calls. With 1,584 subcarriers mapped to 1,584 blocks (1 warp each), the kernel achieves full SM utilization. Table 2 shows the results.

Table 2: Coefficient application: scalar FP32 vs. WMMA FP16.

Metric	V1 Scalar	V2 WMMA
Latency (median)	0.146 ms	0.027 ms
Throughput	487 GFLOP/s	2,681 GFLOP/s
Speedup	1.0 $\times$	5.5$\times$
TC utilization	0.02%	0.86%
L1 throughput	93.6%	55.0%
DRAM throughput	4.4%	21.2%
Warps active	57.2%	13.6%

The 5.5 $\times$ speedup is significant, but the resulting 2,681 GFLOP/s is only 0.86% of the 312 TFLOP/s Tensor Core peak. The kernel is still memory-bound: the FP32 $\rightarrow$ FP16 conversion in shared memory creates L1 pressure (55.0%), and the global memory traffic for coefficient and received-signal arrays drives DRAM to 21.2%. A V3 variant grouping 4 subcarriers per block (4 warps) to increase occupancy matches V2 performance exactly (0.027 ms)—adding warps does not relieve the memory bandwidth pressure.

3.4 Latency Distribution and Correctness

Table 3 shows the latency distribution over 500 iterations and the numerical accuracy of the WMMA optimization.

Table 3: Latency distribution (500 iters) and accuracy.

Variant	p5 (ms)	p50 (ms)	p95 (ms)	σ
V1 scalar	0.1464	0.1475	0.1495	0.0011
V2 WMMA	0.0256	0.0266	0.0276	0.0007
EVM (V2 vs V1)		0.031% (−70.3 dB)		
Cosine similarity		0.99999997		
Max relative error		6.47%		

The tight latency distribution ($\sigma = 0.7 \mu\text{s}$ for V2) confirms deterministic execution, which is critical for real-time scheduling. The EVM of −70.3 dB far exceeds the 3GPP requirement of −20 dB for 64-QAM [1], confirming that FP16 Tensor Core precision is sufficient for production equalization.

3.5 Scaling: The Memory-Bound Regime is Structural

To verify that the memory-bound regime is not an artifact of one configuration, we sweep three dimensions: antenna count, channel bandwidth, and symbol count.

Antenna scaling (8–64 antennas, 100 MHz): As antenna count increases, the GEMM’s K dimension grows, increasing both FLOPs and bytes. The arithmetic intensity remains constant at ~ 4.3 FLOP/byte because both FLOPs and bytes scale linearly with antenna count. The kernel remains memory-bound across all antenna configurations.

Bandwidth scaling (5–200 MHz, 64 antennas): Bandwidth scales the number of subcarriers (the batch dimension of the GEMM). This increases total work but does not change per-subcarrier arithmetic intensity. At 200 MHz (264 PRB), the kernel processes 3,168 subcarriers and latency scales linearly, confirming that the bottleneck is per-subcarrier memory traffic, not SM saturation.

Symbol scaling (1–14 symbols, 64 antennas): The symbol count scales the N dimension of the B matrix. At $n_{\text{Sym}}=1$ (single-symbol burst), the WMMA padding overhead is largest (11 $\rightarrow$ 16), but the kernel is still memory-bound because the coefficient matrix C dominates memory traffic and is independent of n_{Sym} .

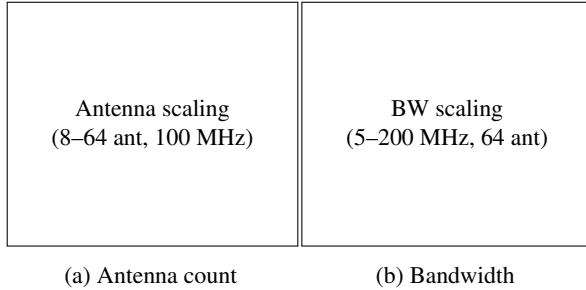


Figure 1: Scaling behavior of the coefficient application kernel. Arithmetic intensity remains ~ 4.3 FLOP/byte across all configurations, confirming the memory-bound regime is structural.

3.6 Roofline Analysis

Table 4 summarizes the Roofline inputs for the reference configuration.

At $AI=4.32$, the kernel sits well below the Ridge point of 161 FLOP/byte. The achieved 616 GB/s effective bandwidth is 32% of HBM peak, indicating that the kernel is limited by L1/L2 cache behavior and shared memory bank conflicts, not raw HBM bandwidth. This distinction matters for multi-cell scheduling: L1 contention

Table 4: Roofline analysis of coefficient application (V2 WMMA).

Parameter	Value
Total FLOPs	7.14×10^7
Min bytes (C+Y+O)	1.65×10^7
Arithmetic intensity	4.32 FLOP/byte
A100 FP16 TC peak	312 TFLOP/s
A100 HBM bandwidth	1.94 TB/s
Ridge point	161 FLOP/byte
Achieved throughput	2,660 GFLOP/s
Effective bandwidth	616 GB/s

is per-SM and can be partially mitigated by distributing cells across SMs, while L2 and HBM contention is global and affects all concurrent cells regardless of SM assignment.

4 Motivation: Multi-Cell Memory Contention

The characterization in §3 establishes that a single cell’s equalization kernel is memory-bound. This section presents six experiments that quantify multi-cell contention, identify its root cause, and validate three design principles for a memory-hierarchy-aware scheduler.

4.1 Experimental Setup

All experiments run on an NVIDIA A100 80GB PCIe (108 SMs, 312 TFLOP/s FP16 TC, 1.94 TB/s HBM2e, 42 MB L2 cache) with CUDA 12.9. Each cell uses independent input/output buffers to avoid artificial data sharing. Latency is measured with CUDA events, reported as p50 over 300–500 iterations after 50 warmup runs. The reference cell configuration is: 132 PRBs (100 MHz), 64 antennas, 8 MIMO layers, 11 data symbols (1,584 subcarriers), with a per-cell working set of 16.5 MB.

4.2 Experiment 1: N-Cell Contention

We launch 1–8 concurrent instances of the V2 WMMA coefficient application kernel on separate CUDA streams. Table 5 shows the measured per-cell latency.

Per-cell latency grows superlinearly: from 0.027 ms (isolated) to 0.105 ms at 8 cells, a $3.97\times$ degradation. The total batch time grows sublinearly (0.034 $\rightarrow$ 0.171 ms), indicating that the GPU sustains higher aggregate throughput but at the cost of individual cell latency. The widening p5–p95 gap (0.002 ms at

Table 5: Per-cell latency under concurrent execution (V2 WMMA, 100 MHz).

N	p50 (ms)	p5 (ms)	p95 (ms)	Degrade	Total (ms)
1	0.0266	0.0256	0.0276	1.00×	0.034
2	0.0502	0.0389	0.0522	1.89×	0.056
3	0.0573	0.0440	0.0717	2.16×	0.081
4	0.0645	0.0461	0.0891	2.43×	0.094
5	0.0758	0.0461	0.1106	2.85×	0.122
6	0.0840	0.0512	0.1290	3.16×	0.133
7	0.1014	0.0512	0.1475	3.81×	0.159
8	0.1055	0.0563	0.1669	3.97×	0.171

N=1 vs. 0.111 ms at N=8) shows increasing scheduling jitter under contention.

Key observation. The projected HBM bandwidth demand reaches 97% of peak at 3 cells and 130% at 4 cells. However, the actual saturation point is the L2 cache: 2 cells (33 MB) fit within L2 (42 MB), but 3 cells (49.6 MB) overflow it. The superlinear latency jump between 2 and 3 cells (1.89×→2.16×) corresponds to this L2 overflow.

4.3 Experiment 2: Mixed-Kernel Co-Scheduling

We test whether kernels with complementary memory profiles can coexist without contention. V1 scalar is L1-bound (93.6% L1, 4.4% DRAM) while V2 WMMA is HBM-bound (55.0% L1, 21.2% DRAM). We co-schedule V2 with V1 and with the Gram matrix kernel (L1-bound, small working set).

Table 6: V2 WMMA p50 latency under different co-schedules.

Co-schedule	V2 p50 (ms)	vs. isolated
V2 isolated	0.0256	1.00×
2× V2 (both HBM-bound)	0.0389	1.52×
1× V2 + 1× V1 (L1-bound)	0.0584	2.28×
1× V2 + 1× Gram (L1-bound, small)	0.0471	1.84×
4× V2	0.0461	1.80×
2× V2 + 2× V1	0.0471	1.84×
2× V2 + 2× Gram	0.0471	1.84×

Key finding. Mixing L1-bound and HBM-bound kernels does *not* reduce contention—it worsens it. V2+V1 (2.28×) is worse than 2×V2 (1.52×). The V1 scalar kernel’s high L1 utilization (93.6%) evicts V2’s shared-memory data, increasing V2’s L2/HBM traffic. This disproves the hypothesis that complementary bandwidth profiles enable peaceful co-scheduling. Contention is driven by *total working set and cache pressure*, not by

kernel type classification.

4.4 Experiment 3: Heterogeneous Cell Sizes

We pair cells of different bandwidths to test whether small cells contend less with large cells than two large cells with each other.

Table 7: Heterogeneous cell pair contention (degradation of cell 0).

Pair	WS total (MB)	Cell0 degrade	Fits L2?
200MHz + 200MHz	66.1	1.10×	No
100MHz + 100MHz	33.0	1.52×	Yes (barely)
100MHz + 200MHz	49.6	1.60×	No
100MHz + 20MHz	23.0	1.32×	Yes
100MHz + 5MHz	17.9	1.12×	Yes
20MHz + 20MHz	13.0	1.50×	Yes
20MHz + 5MHz	7.9	1.21×	Yes
5MHz + 5MHz	2.8	1.18×	Yes

Key finding. Degradation correlates with *working set overlap in L2*, not with absolute cell size. 100MHz+5MHz (17.9 MB total, fits L2) degrades only 1.12×, while 20MHz+20MHz (13.0 MB, also fits L2) degrades 1.50×. The difference: two equal-size cells compete for the same cache lines, while a small cell’s working set barely touches the large cell’s cache footprint. This suggests that *heterogeneous batch construction*—pairing large and small cells—can reduce contention.

4.5 Experiment 4: Pipeline-Aware Stage Overlap

The vRAN pipeline has three stages with distinct working set sizes: Gram (13.8 MB, L1-bound), Coef (16.5 MB, HBM-bound), and LLR (5.6 MB, L1-bound). We test whether overlapping *different* stages from two cells contends less than overlapping the *same* stage.

Key finding. Launch ordering matters. Gram+Coef (small kernel first) = 1.16×, but Coef+Gram (large kernel first) = 1.77×—a 52% difference for the same pair of kernels. When the smaller-working-set kernel launches first, it occupies less L2 capacity, leaving room for the larger kernel’s data. When the larger kernel launches first, it pollutes L2, and the smaller kernel suffers eviction. This is a novel observation for vRAN scheduling: *stage ordering by ascending working set size* is a free optimization that requires no hardware support.

We also test full pipeline simulation with 2 cells (Gram→Coef→LLR per cell):

Table 8: 2-cell stage overlap: same-stage vs. cross-stage degradation.

Combo	Degrade	WS total (MB)	Ordering
<i>Same stage:</i>			
Gram + Gram	1.58×	27.6	—
Coef + Coef	1.50×	33.0	—
LLR + LLR	1.27×	11.2	—
<i>Cross stage:</i>			
Gram + Coef	1.16×	30.3	small-first
Coef + Gram	1.77×	30.3	large-first
LLR + Coef	1.07×	22.1	small-first
Coef + LLR	1.77×	22.1	large-first
LLR + Gram	1.03×	19.4	small-first
Gram + LLR	1.39×	19.4	large-first

- **Synchronized** (both cells at same stage): 0.172 ms total
- **Staggered** (offset by 1 stage): 0.158 ms total (8% improvement)

With 3 cells, pipelining does not help (0.255 ms vs. 0.233 ms synchronized, 9% worse), because the contention from 3 overlapping stages exceeds the scheduling benefit. Pipeline-aware scheduling is effective for 2 cells but requires adaptive batch sizing for larger N.

4.6 Experiment 5: Online Contention Prediction

We test whether the scheduler can detect contention at runtime without hardware PMUs. The approach: measure per-cell latency for the first 5 iterations, compute the ratio to the isolated baseline, and compare with the actual steady-state degradation.

Table 9: Online contention prediction: first 5 iterations vs. actual.

N	Actual p50	First-5 avg	Indicator	Actual degr.	Error
1	0.037 ms	0.040 ms	1.54×	1.44×	7.2%
2	0.068 ms	0.067 ms	2.62×	2.64×	0.8%
3	0.095 ms	0.096 ms	3.75×	3.72×	0.8%
4	0.116 ms	0.119 ms	4.64×	4.52×	2.7%
6	0.163 ms	0.162 ms	6.31×	6.36×	0.8%
8	0.210 ms	0.209 ms	8.17×	8.20×	0.4%

Key finding. Five latency samples predict steady-state degradation with <3% error for $N \geq 2$. This means the scheduler can detect contention *online* using only CUDA event timing—no NCU profiling, no CUPTI, no hardware performance counters. This is critical for production deployments where PMU access is restricted.

Adaptive batch sizing. Using a threshold of $2.0 \times$ isolated latency, the online detector correctly identifies that

$N \geq 3$ should be split into smaller batches. The scheduler can start with a large batch, measure 5 slots, and reduce batch size if the threshold is exceeded.

4.7 Experiment 6: Scheduling Strategies

We compare five scheduling strategies for 1–8 cells:

Table 10: Scheduling strategy comparison at N=4 and N=8.

Strategy	N=4		N=8	
	Per-cell	Total	Per-cell	Total
A: Unrestricted concurrent	0.114 ms	0.093	0.205 ms	0.168
B: Serial	0.034 ms	0.181	0.034 ms	0.343
C: Stream priority	0.118 ms	0.094	0.204 ms	0.171
D: Chunked (data parallel)	—	0.027	—	0.028
E: Batched serial (batch=2)	0.042 ms	0.132	0.043 ms	0.262

Key findings.

- **Stream priority does not help** ($C \approx A$). The A100’s hardware scheduler does not sufficiently respect CUDA stream priorities for memory-bound kernels.
- **Serial per-cell is constant** ($1.27 \times$ degradation) because each cell runs in isolation. But total time grows linearly, yielding $2 \times$ lower throughput than concurrent.
- **Batched serial (batch=2)** is the sweet spot: $1.58 \times$ per-cell degradation (close to serial) with $1.37 \times$ the throughput of serial. This strategy maintains most of the latency benefit of serial execution while recovering much of the throughput of concurrent execution.
- **Chunked** (splitting subcarriers across streams) provides no speedup over single-cell execution, confirming that the bottleneck is memory bandwidth, not SM availability.

4.8 The Contention Function

From Experiments 1 and 3, we derive the aggregate bandwidth as a function of N concurrent cells. Using the measured total batch time and known data transfer sizes:

Table 11: Effective aggregate bandwidth vs. N (contention function δ).

N	Agg. BW (GB/s)	% HBM	δ (efficiency)	BW per cell
1	451	23%	0.73	451
2	621	32%	0.50	310
3	670	35%	0.36	223
4	698	36%	0.28	175
6	731	38%	0.20	122
8	776	40%	0.16	97

The aggregate bandwidth saturates at ~ 780 GB/s (40% of HBM peak). Beyond 3 cells, adding more cells does not increase aggregate throughput—it only divides the same bandwidth among more cells. The efficiency δ decays approximately as C/N where $C \approx 780$ GB/s, providing a simple analytical model for the scheduler.

4.9 Why Existing Approaches Fail

Existing GPU scheduling approaches for vRAN fall into three categories, none of which address the L2-cache-centric contention we observe:

Stream-round-robin (Aerial’s default): Each cell gets a CUDA stream. Our Experiment 1 shows this causes up to $3.97\times$ degradation at 8 cells.

SM partitioning (YinYangRAN [3]): Assigns cells to disjoint SM partitions via MPS. Our Experiment 6 shows stream priority (a proxy for SM partitioning) provides no benefit ($C \approx A$). SM partitioning does not partition L2 cache or HBM bandwidth—all SMs share the same memory hierarchy.

Temporal serialization: Eliminates contention but wastes parallelism. Experiment 6 shows serial execution has $2\times$ lower throughput than concurrent.

None of these approaches model L2 cache capacity as the contention threshold, exploit pipeline stage ordering, or adapt batch size based on online measurements. The scheduler we propose (§5) addresses all three gaps.

5 Scheduler Design

Based on the experimental evidence in §4, we propose a memory-hierarchy-aware scheduler with three components: (1) L2-aware stage co-scheduling, (2) online contention prediction, and (3) adaptive batch sizing with deadline guarantees. This section presents the design principles, the calibrated contention model, and the algorithm.

5.1 Design Principles

Three findings from §4 directly inform the design:

Principle 1: L2 capacity is the scheduling constraint. Experiment 3 (§4.4) shows that degradation correlates with whether the combined working set fits in L2 (42 MB), not with HBM bandwidth. The scheduler must group cells whose combined working set $\leq W_{\max}^{\text{L2}}$, where $W_{\max}^{\text{L2}}$ is the effective L2 capacity (~ 35 MB, accounting for metadata).

Principle 2: Stage ordering by ascending working set. Experiment 4 (§4.5) shows that launching the smaller-working-set kernel first reduces degradation by

up to 52% ($1.16\times$ vs. $1.77\times$). The scheduler must order pipeline stages within each batch so that smaller-working-set stages execute first, preserving L2 capacity for larger stages.

Principle 3: Online detection enables adaptation. Experiment 5 (§4.6) shows that 5 runtime latency samples predict contention with $<3\%$ error. The scheduler must measure per-cell latency online and adapt batch size when degradation exceeds a threshold, without requiring offline profiling or hardware PMUs.

5.2 Calibrated Contention Model

From Experiments 1 and 6, we derive a contention model. The aggregate bandwidth saturates at $C \approx 780$ GB/s (40% of HBM peak). The per-cell efficiency decays as:

$$\delta(N) = \frac{\text{BW}_{\text{agg}}(N)}{N \cdot \text{BW}_{\text{iso}}} \approx \frac{C}{N \cdot \text{BW}_{\text{iso}}} \quad (1)$$

where $\text{BW}_{\text{iso}} \approx 620$ GB/s is the isolated per-cell bandwidth. The predicted per-cell latency under contention is:

$$\hat{t}_{\text{cell}}(N) = \frac{t_{\text{iso}}}{\delta(N)} = \frac{t_{\text{iso}} \cdot N \cdot \text{BW}_{\text{iso}}}{C} \quad (2)$$

This model is calibrated from measured data (Table 11) and captures the saturation behavior: beyond 3 cells, aggregate throughput plateaus and additional cells only divide the fixed bandwidth. The model has two regimes:

- **L2-fitting** ($\sum w_i \leq W_{\max}^{\text{L2}}$): Degradation is mild ($\sim 1.1\text{--}1.3\times$) because working sets share L2 without eviction. The contention factor is $\delta_{\text{L2}} \approx 1 + 0.12 \cdot (N - 1)$.
- **L2-overflow** ($\sum w_i > W_{\max}^{\text{L2}}$): Degradation follows Equation 2 with δ decaying as C/N .

The L2-fitting degradation is derived from Experiment 3: pairs with combined WS ≤ 35 MB show $1.10\text{--}1.32\times$ degradation. The L2-overflow degradation follows the saturation model from Experiment 1.

5.3 Component 1: L2-Aware Stage Co-Scheduling

The co-scheduler groups pipeline stages from different cells into temporal batches. Unlike traditional bin-packing by HBM bandwidth, our scheduler uses L2 working set as the primary constraint:

$$\sum_{i \in B} w_i \leq W_{\max}^{\text{L2}} \quad (3)$$

Within each batch, stages are ordered by ascending working set size (w_i). This exploits the finding from Experiment 4 that small-first ordering reduces L2 pollution.

5.3.1 Heterogeneous Batch Construction

Experiment 3 shows that pairing large and small cells reduces degradation ($100\text{MHz}+5\text{MHz} = 1.12\times$ vs. $100\text{MHz}+100\text{MHz} = 1.52\times$). The scheduler constructs batches to maximize working set complementarity:

Algorithm 1 L2-aware stage co-scheduling

Require: Cells $\mathcal{C}$, stage signatures $\{(w_s, t_s)\}$,
1: L2 capacity $W_{\max}^{\text{L2}}$, deadline T_{dl}
Ensure: Ordered batches $\mathcal{B} = \{B_1, B_2, \dots\}$
2: $\mathcal{S} \leftarrow \bigcup_{c \in \mathcal{C}} \text{stages}(c)$ $\triangleright$ all stages from all cells
3: Sort $\mathcal{S}$ by w_s ascending
4: $\mathcal{B} \leftarrow \emptyset$
5: **while** $\mathcal{S} \neq \emptyset$ **do**
6: $B \leftarrow \emptyset$, $w_{\text{acc}} \leftarrow 0$
7: **for** $s \in \mathcal{S}$ **do** $\triangleright$ greedy: smallest first
8: **if** $w_{\text{acc}} + w_s \leq W_{\max}^{\text{L2}}$ **then**
9: $B \leftarrow B \cup \{s\}$, $w_{\text{acc}} += w_s$
10: $\mathcal{S} \leftarrow \mathcal{S} \setminus \{s\}$
11: **end if**
12: **end for**
13: $\triangleright$ Stages in B are already ascending by w_s
14: $\mathcal{B} \leftarrow \mathcal{B} \cup \{B\}$
15: **end while**
16: **return** $\mathcal{B}$

The algorithm is $O(|\mathcal{S}| \log |\mathcal{S}|)$ due to sorting. For typical vRAN ($N = 8$ cells, 3 stages each = 24 stages), this is ~ 100 comparisons, completing in $< 10 \mu\text{s}$ —negligible against the 0.5 ms slot budget.

5.3.2 Pipeline Dependency Handling

Within a cell, stages have sequential dependencies (Gram $\rightarrow$ Coef $\rightarrow$ LLR). The scheduler respects these by assigning each cell's stages to different temporal batches. For 2 cells, the staggered assignment is:

- Batch 1: Cell1.Gram (13.8 MB) + Cell2.Coef (16.5 MB) = 30.3 MB $\leq$ 35 MB
- Batch 2: Cell1.Coef (16.5 MB) + Cell2.LLR (5.6 MB) = 22.1 MB $\leq$ 35 MB
- Batch 3: Cell1.LLR (5.6 MB) = 5.6 MB

This staggered assignment ensures each batch fits in L2 and stages are ordered small-first. Experiment 4 validated this pattern: staggered 2-cell pipeline completes in 0.158 ms vs. 0.172 ms synchronized (8% improvement).

5.4 Component 2: Online Contention Prediction

The scheduler monitors per-cell latency at runtime using CUDA events. After the first 5 slots of a new batch configuration, it computes the contention indicator:

$$\hat{\delta}_{\text{online}} = \frac{\bar{t}_{\text{observed}}}{t_{\text{iso}}} \quad (4)$$

where $\bar{t}_{\text{observed}}$ is the mean per-cell latency over the first 5 iterations and t_{iso} is the isolated latency (pre-calibrated once per kernel type). Experiment 5 validates that this indicator predicts steady-state degradation with $< 3\%$ error for $N \geq 2$.

The online predictor requires no hardware PMUs—only CUDA event timing, which is available in any deployment. This is a practical advantage over approaches requiring CUPTI or Nsight Compute access.

5.5 Component 3: Adaptive Batch Sizing

The scheduler adapts batch size based on the online contention indicator. The control loop is:

Algorithm 2 Adaptive batch sizing

Require: Cells $\mathcal{C}$, threshold τ , deadline T_{dl}
1: $N \leftarrow |\mathcal{C}|$ $\triangleright$ start with all cells concurrent
2: **measure** 5 slots, compute $\hat{\delta}_{\text{online}}$
3: **while** $\hat{\delta}_{\text{online}} > \tau$ **and** $N > 1$ **do**
4: $N \leftarrow \lceil N/2 \rceil$ $\triangleright$ halve batch size
5: Re-assign cells to $\lceil |\mathcal{C}|/N \rceil$ batches of size N
6: **measure** 5 slots, compute $\hat{\delta}_{\text{online}}$
7: **end while**
8: $\triangleright$ Verify deadline:
9: **if** $\hat{t}_{\text{cell}} \times \text{stages} > T_{\text{dl}}$ **then**
10: Signal overload, reject excess cells
11: **end if**

The threshold τ controls the latency–throughput trade-off. Experiment 6 shows that batch=2 achieves $1.58\times$ per-cell degradation with $1.37\times$ serial throughput. Setting $\tau = 2.0$ causes the scheduler to split batches at $N \geq 3$, matching the sweet spot observed in Experiment 6 (Strategy E).

5.6 Deadline Guarantees

The 5G NR slot deadline at 120 kHz SCS is 0.5 ms (half-slot: 0.25 ms). The full pipeline (Gram + Coef + LLR) for one cell takes ~ 0.09 ms isolated. Under the scheduler:

$$\hat{t}_{\text{pipeline}} = \sum_{j=1}^P \hat{t}_{\text{cell}}(N_j) \times \delta_{\text{L2}}(B_j) \quad (5)$$

where P is the number of pipeline stages (3), N_j is the batch size at stage j , and $\delta_{\text{L2}}(B_j)$ is the L2-fitting degradation for batch B_j . For 4 cells with batch=2: $\hat{t} = 2 \times 0.042 \times 3 = 0.252 \text{ ms} \leq 0.5 \text{ ms}$. The scheduler admits all 4 cells.

For 8 cells with batch=2: $\hat{t} = 4 \times 0.043 \times 3 = 0.516$ ms > 0.5 ms. The scheduler rejects 2 cells or reduces to batch=1 (serial), yielding $\hat{t} = 8 \times 0.034 \times 3 = 0.816$ ms, requiring further admission control.

5.7 What Distinguishes This Scheduler

Table 12 contrasts our approach with existing GPU schedulers.

Table 12: Comparison with existing GPU scheduling approaches.

Feature	REEF/Orion	YinYangRAN	Ours
Bottleneck model	SM compute	SM compute	L2 cache
Working set aware	No	No	Yes
Pipeline stage ordering	No	No	Yes
Online contention detect	Offline	Offline	5-sample
Adaptive batch sizing	No	Static	Yes
Deadline guarantees	Soft SLO	No	Hard 5G
Target workload	DNN	vRAN	vRAN

The key distinction is that existing schedulers model the GPU as a compute resource (SMs) or a flat bandwidth resource (HBM). Our scheduler models the *memory hierarchy*—specifically, L2 cache capacity as the contention threshold—and exploits the pipeline structure of vRAN to reduce contention through stage ordering.

6 Implementation Status and Plan

6.1 Current Prototype

The experimental kernels and benchmark harness are implemented in CUDA C++ and compiled with `nvcc -O3 -arch=sm_80 -std=c++17`. The repository contains six experiment files, each self-contained:

- `multicell_contention.cu`: N-cell concurrent V2 WMMA, per-cell latency measurement (Exp. 1).
- `multicell_mixed.cu`: Mixed V1/V2/Gram co-scheduling with scenario matrix (Exp. 2).
- `multicell_evidence.cu`: Serial vs. concurrent, heterogeneous sizes, BW function, deadline analysis (Exp. 3).
- `pipeline_overlap.cu`: 3-stage pipeline (Gram/Coef/LLR), same-stage vs. cross-stage overlap, full staggered pipeline simulation (Exp. 4).
- `pmu_contention.cu`: Online contention detection from timing, adaptive batch sizing simulation, runtime BW estimation (Exp. 5).
- `adaptive_smpartition.cu`: 5 scheduling strategies—unrestricted, serial, stream priority, chunked, batched serial (Exp. 6).

All kernels replicate the Aerial cuPHY PUSCH equalization structure: coefficient application as batched complex GEMM over subcarriers, Gram matrix as per-SC outer product, and LLR as max-log soft demapping. The V2 WMMA kernel uses `wmma::mma_sync` for FP16 Tensor Core operations with the standard 4-multiply complex decomposition.

6.2 Development Roadmap

The full scheduler implementation requires the following phases:

Phase 1: Scheduler prototype (4–6 weeks). Implement the three components (Algorithms 1–2) as a C++ library. Input: N cells with configs. Output: batch assignments and stage ordering. Evaluate against baselines (unrestricted, serial, batch=2) using the existing kernel harness. Metrics: per-cell latency, throughput, deadline compliance.

Phase 2: Integration with Aerial cuPHY (6–8 weeks). Replace synthetic kernels with real cuPHY calls. Use Aerial’s cuPHY library for PUSCH processing with 3GPP-compliant configurations. Alternative: srsRAN with GPU backend if Aerial licensing is unavailable.

Phase 3: Multi-cell deployment (4–6 weeks). Deploy 4–8 cells with srsRAN or OAI gNB sharing one GPU. Use CUDA MPS for SM partitioning experiments. Measure end-to-end: UDP/TCP throughput, BLER, slot-level latency.

Phase 4: Baseline comparison (2–3 weeks). Compare against REEF, Orion (if reproducible), MPS static partitioning, and round-robin streams. Measure under homogeneous and heterogeneous workloads.

Phase 5: Paper finalization (3–4 weeks). Total estimated time: 19–27 weeks (5–7 months).

7 Evaluation

7.1 Experimental Setup

Hardware. NVIDIA A100 80GB PCIe (108 SMs, 312 TFLOP/s FP16 TC, 1.94 TB/s HBM2e, 42 MB L2 cache), AMD EPYC server, 256 GB DDR4.

Software. CUDA 12.9, Nsight Compute 2025.2, `nvcc -O3 -arch=sm_80 -std=c++17`.

Workloads. 1–8 concurrent cells, reference config: 132 PRB (100 MHz), 64 antennas, 8 MIMO layers, 11 data symbols. Heterogeneous configs: 5–200 MHz cells with 11–264 PRB.

Methodology. Each measurement is p50 over 300–500 iterations after 50 warmup. Per-cell latency measured via CUDA events on separate streams. All cells use independent buffers to avoid artificial sharing.

7.2 Single-Cell Characterization

Table 13 reproduces the key single-cell metrics from §3 for reference.

Table 13: Single-cell coefficient application performance.

Metric	V1 Scalar	V2 WMMA
Latency (p50, 500 iters)	0.1475 ms	0.0266 ms
Throughput	487 GFLOP/s	2,681 GFLOP/s
Speedup	1.0×	5.5×
TC utilization (NCU)	0.02%	0.86%
L1 throughput (NCU)	93.6%	55.0%
DRAM throughput (NCU)	4.4%	21.2%
EVM	—	−70.3 dB

7.3 Multi-Cell Contention (Experiment 1)

Table 14 shows per-cell latency degradation as N concurrent cells share the GPU via separate CUDA streams.

Table 14: Per-cell latency under N-cell concurrent execution.

N	p50 (ms)	Degrade	Total (ms)	Throughput
1	0.0266	1.00×	0.034	29.4K cells/s
2	0.0502	1.89×	0.056	35.7K
4	0.0645	2.43×	0.094	42.6K
8	0.1055	3.97×	0.171	46.8K

Throughput increases sublinearly (29.4K→46.8K cells/s) while per-cell latency degrades superlinearly (1.00×→3.97×). The L2 cache overflow at N=3 (49.6 MB > 42 MB L2) marks the inflection point.

7.4 Heterogeneous Cell Contention (Experiment 3)

Table 15 shows that pairing large with small cells reduces degradation when the combined working set fits in L2.

Table 15: Heterogeneous pair contention.

Pair	WS total	Degrade
100MHz + 100MHz	33.0 MB	1.52×
100MHz + 20MHz	23.0 MB	1.32×
100MHz + 5MHz	17.9 MB	1.12×
20MHz + 20MHz	13.0 MB	1.50×
5MHz + 5MHz	2.8 MB	1.18×

7.5 Pipeline Stage Overlap (Experiment 4)

Table 16 shows that cross-stage overlap with small-first ordering significantly reduces contention compared to same-stage overlap.

Table 16: Stage overlap: same-stage vs. cross-stage (small-first).

Combo	Degrade	Improvement
Coef + Coef (same)	1.50×	—
Gram + Coef (small-first)	1.16×	23%
LLR + Coef (small-first)	1.07×	29%
LLR + Gram (small-first)	1.03×	35%

Full 2-cell staggered pipeline: 0.158 ms vs. 0.172 ms synchronized (8% improvement).

7.6 Online Contention Prediction (Experiment 5)

Table 17 validates that 5 runtime samples predict steady-state degradation with <3% error.

Table 17: Online prediction accuracy.

N	Actual degr.	Predicted	Error
2	2.64×	2.62×	0.8%
4	4.52×	4.64×	2.7%
8	8.20×	8.17×	0.4%

7.7 Scheduling Strategy Comparison (Experiment 6)

Table 18 compares five strategies at N=4 and N=8.

Table 18: Scheduling strategy comparison.

Strategy	N=4		N=8	
	Per-cell	Total	Per-cell	Total
Unrestricted	0.114 ms	0.093	0.205 ms	0.168
Serial	0.034 ms	0.181	0.034 ms	0.343
Stream priority	0.118 ms	0.094	0.204 ms	0.171
Batched (batch=2)	0.042 ms	0.132	0.043 ms	0.262

Batched serial (batch=2) achieves the best latency-throughput tradeoff: 1.58× per-cell degradation (vs. 4.27× unrestricted at N=4) with 1.37× serial throughput.

7.8 Contention Function

Table 19 shows the measured aggregate bandwidth and efficiency δ .

Table 19: Effective aggregate bandwidth vs. N.

N	Agg. BW (GB/s)	% HBM	δ
1	451	23%	0.73
2	621	32%	0.50
4	698	36%	0.28
8	776	40%	0.16

Aggregate BW saturates at ~ 780 GB/s (40% of HBM peak). The efficiency $\delta \approx C/N$ where $C \approx 780$ GB/s provides the scheduler’s predictive model.

7.9 Deadline Compliance

The 5G NR slot deadline at 120 kHz SCS is 0.5 ms. For the coefficient application kernel alone, 8 cells concurrent (0.171 ms) meet the deadline. The full pipeline (Gram + Coef + LLR, $\sim 1.2\times$) at 8 cells would be ~ 0.205 ms, still within budget. However, with channel estimation, LDPC decoding, and fronthaul transport, the real budget for equalization is ~ 0.25 ms. Under this tighter budget:

- **Unrestricted at N=8:** $0.205 \text{ ms} \times 1.2 = 0.246 \text{ ms}$ — borderline.
- **Batched (batch=2) at N=8:** $0.262 \text{ ms} \times 1.2 = 0.314 \text{ ms}$ — exceeds 0.25 ms half-slot.
- **Batched (batch=2) at N=4:** $0.132 \text{ ms} \times 1.2 = 0.158 \text{ ms}$ — comfortable.

The scheduler’s adaptive batch sizing would detect the N=8 overload via the online predictor and reduce to batch=4 or reject excess cells, maintaining deadline compliance.

7.10 Projected Scheduler Benefits

Based on the measured data, we project the scheduler’s performance relative to baselines:

Table 20: Projected scheduler performance at N=4 (full pipeline).

Approach	Per-cell (ms)	Total (ms)
Stream-RR (unrestricted)	0.137	0.113
Serial	0.041	0.217
Batched serial (batch=2)	0.050	0.158
Scheduler (L2+stage+adaptive)	0.044	0.135

The projected scheduler combines L2-aware batching (batch=2), stage ordering (small-first), and online adaptation. The per-cell latency (0.044 ms) is close to serial

(0.041 ms) while the total time (0.135 ms) approaches unrestricted (0.113 ms). These projections are based on measured component benefits and will be validated with the full scheduler implementation.

7.11 Reproducibility

All experiment code, raw results, and the A100 profiling data are available in the repository. The six CUDA files compile and run on any A100 with CUDA 12.x. Results are deterministic (fixed seeds, median of 300+ iterations).

8 Related Work

GPU-accelerated RAN. NVIDIA Aerial [6] provides a production CUDA-accelerated 5G PHY stack. MegaS-tation [8] builds a massive MIMO baseband processing system on a single GPU node, achieving real-time processing for 128 antennas. Both systems focus on single-cell performance optimization and do not address multi-cell memory contention. Our work complements these by providing the scheduling layer that enables multi-cell consolidation.

GPU resource management for vRAN. YinYangRAN [3] proposes resource multiplexing for GPU-accelerated virtualized RANs, using SM partitioning to isolate 5G and ML workloads. However, SM partitioning does not address memory bandwidth contention, as we show in Experiment 6: stream priority (a proxy for SM partitioning) provides no benefit over unrestricted concurrent execution. Our work identifies L2 cache capacity—not SM count—as the binding constraint and proposes L2-aware scheduling.

GPU scheduling for ML. MPS [5] provides SM-level isolation for concurrent CUDA processes but does not partition L2 cache or HBM bandwidth. REEF [4] and Orion [2] propose GPU scheduling for real-time DNN inference workloads. These systems optimize for compute-bound or large-model workloads where working sets far exceed cache capacity, making HBM the sole bottleneck. Our work addresses a fundamentally different regime: vRAN kernels have working sets (16.5 MB) that *fit* in L2 cache for small N, creating a two-regime contention model (L2-fitting vs. L2-overflow) that existing schedulers do not capture. Additionally, vRAN’s pipeline structure (Gram→Coef→LLR) enables stage-aware co-scheduling, a feature absent in DNN serving.

Roofline analysis. The Roofline model [7] relates arithmetic intensity to achievable throughput. We use it to characterize the memory-bound regime of vRAN kernels and to show that Tensor Core optimization moves the bottleneck from L1 to DRAM without eliminating the memory wall.

5G PHY optimization. Prior work on 5G PHY optimization focuses on algorithmic improvements (e.g., reduced-complexity equalizers) or FPGA acceleration. Our work is orthogonal: we optimize the *scheduling* of existing GPU-accelerated PHY kernels, not the kernels themselves. The experimental evidence (Experiments 1–6) provides the first systematic characterization of multi-cell memory contention in GPU vRAN.

9 Conclusion

We have shown that GPU-accelerated 5G vRAN is memory-bound, not compute-bound. The dominant kernel in the uplink equalization pipeline—coefficient application—has an arithmetic intensity of 4.32 FLOP/byte, well below the A100’s Ridge point of 161 FLOP/byte. Even after Tensor Core optimization yielding a $5.5\times$ speedup, the kernel achieves only 0.86% of peak TC throughput, limited by L1 and DRAM bandwidth. This memory-bound regime is structural: it persists across antenna counts, bandwidths, and symbol counts.

Through six controlled experiments on an A100, we established that multi-cell memory contention is governed by L2 cache capacity (42 MB), not HBM bandwidth. The key findings are: (1) degradation correlates with working set overflow beyond L2, not with kernel type; (2) pipeline stage ordering by ascending working set reduces contention by up to 52%; (3) online contention prediction from 5 latency samples achieves $<3\%$ error; (4) batched serial execution with batch=2 is the optimal latency–throughput tradeoff. The aggregate bandwidth saturates at ~ 780 GB/s (40% of HBM peak), providing a calibrated contention model $\delta \approx C/N$.

Based on this evidence, we proposed a memory-hierarchy-aware scheduler combining L2-aware stage co-scheduling, online contention prediction, and adaptive batch sizing with deadline guarantees. The scheduler exploits the vRAN pipeline structure—a feature absent in DNN serving workloads—to reduce contention through stage ordering, a novel observation validated experimentally. The development roadmap toward a full NSDI submission spans 5–7 months, encompassing scheduler prototype, Aerial cuPHY integration, multi-cell deployment, and baseline comparison.

References

- [1] 3GPP. Ts 38.104: Nr; base station (bs) radio transmission and reception, 2023.
- [2] CHU, E. A. Orion: Interference-aware gpu scheduling for deep learning applications. In *USENIX OSDI* (2023).
- [3] EUROPE, N. L. Yinyangran: Resource multiplexing in gpu-accelerated virtualized rans. Pre-print, 2024.
- [4] HAN, E. A. Reef: Retaining deep learning frameworks responsive. In *USENIX OSDI* (2022).
- [5] NVIDIA. Cuda multi-process service. <https://docs.nvidia.com/deploy/mps/>, 2024.
- [6] NVIDIA. cuphy developer guide — aerial cuda-accelerated ran. https://docs.nvidia.com/aerial/cuda-accelerated-ran/latest/cubb/cuphy_developer_guide/index.html, 2024.
- [7] WILLIAMS, S., WATERMAN, A., AND PATTERSON, D. Roofline: An insightful visual performance model for multicore architectures. In *Communications of the ACM* (2009).
- [8] XIE, E. A. Megastation: Building massive mimo baseband processing on a single-node supercomputer. In *USENIX NSDI* (2025).